

Foundations of Cybersecurity/Applied Cryptography

19 September 2023

EXERCISE N. 1 [ALL]

[12]

With reference to hash functions, the candidate

- 1) State the preimage resistance property, the 2nd preimage resistance property, and the collision resistance property.
- 2) List the black box attacks specifying, for each of them, the attack complexity.
- 3) Argue why black box attacks constitute a security upper limit.

EXERCISE N. 2 [ALL]

[12]

Let p be a large prime and g be a generator of $(\mathbb{Z}_p^*, \times)$. Suppose we define the hash function $h: \mathbb{Z} \rightarrow \mathbb{Z}_p^*$, where $h(x) = g^x \bmod p$, assuming x to be an arbitrary binary string interpreted as a positive integer number. Assuming the computational difficulty of the discrete logarithm problem in $\mathbb{Z}_p^*$, the candidate should argue whether the hash function satisfies the following properties:

- 1) compression,
- 2) preimage resistance,
- 3) 2nd preimage resistance,
- 4) collision resistance.

EXERCISE N. 3 [2022-2023]

[6]

Discuss the confidentiality property of the following OTP-based communication protocol to transmit a given message m from Alice to Bob:

M1 $A \rightarrow B: c = m \oplus a$

M2 $B \rightarrow A: d = c \oplus b$

M3 $A \rightarrow B: e = d \oplus a$

Upon receiving M3, Bob computes $m = e \oplus b$, with a and b random numbers generated by Alice and Bob, respectively

EXERCISE N. 3 [Earlier than 2022-23]

[6]

Find, explain, and patch possible vulnerabilities in the following function.

```
struct student{
    int StudentID;
    char FirstName[100];
    char LastName[100];
    int DegreeID;
}
//this function fills the structure stu's fields by means of
//input parameters Sid, Did, Fname, Lname
void EditStudent(student* stu, int Sid, int Did, char* Fname, char* Lname)
{
    stu->StudentID = Sid;
    strcpy(stu->FirstName, Fname);
    strcpy(stu->LastName, Lname);
    stu->DegreeID = Did;
}
```

Solution

EXERCISE N.1

See theory.

EXERCISE N.2 [2022-23]

- The *compression property* is satisfied as messages x of arbitrary bit-length are hashed into a fixed size digest.
- The *first pre-image resistance property* is guaranteed by the difficulty of the discrete logarithm problem.
- The *second pre-image resistance property* is not satisfied as, according to the Fermat's Little Theorem, given x and $h(x)$, it is easy to compute $x' = x + k \times (p - 1)$ with $k \in \{1, 2, 3, \dots\}$, such that $h(x) = h(x')$.
- The *collision resistance property* is also not satisfied as two messages x_1 and x_2 with the same digest can be *easily* found by picking them in the set $\{x_2 \mid x_1 + k \times (p - 1)\}$ with $k \geq 1, \forall x_1 \geq 0$. We can exploit again the Fermat's Little Theorem. In addition, we know that collision resistance implies preimage resistance (necessary condition), therefore, if preimage resistance is not guaranteed then collision resistance cannot be guaranteed.

EXERCISE N.3 [Earlier than 2022-23]

The protocol is insecure. Actually, an adversary that eavesdrops communication may easily determine secrets by performing the following computations:

- 1) $a = M2 \oplus M3$.
- 2) $m = M1 \oplus a$.

The proof is easy.

EXERCISE N.3 [Earlier than 2022-23]

There are two vulnerabilities. First, pointer `stu` is not checked against the `NULL` value. Second, a consists in the use of function `strcpy(dest, src)` that has an *undefined behavior* “if either `dest` is not a pointer to a character array or `src` is not a pointer to a null-terminated byte string.”¹ The correct approach is: 1) to use the `strncpy(dest, src, size)` function², with parameter `size` specifying the size of the buffer minus one; and then 2) to insert the null terminator into the last place of the buffer. So, the a possible solution is the following.

```
//this function fills the structure stu's fields by means of
//input parameters Sid, Did, Fname, Lname
void EditStudent(student* stu, int Sid, int Did, char* Fname, char* Lname)
{
    If (stu != NULL) {
        stu->StudentID = Sid;
        strncpy(stu->FirstName, Fname, 99);
        stu->FirstName[99]='\0';
        strncpy(stu->LastName, Lname, 99);
        stu->LastName[99]='\0';
        stu->DegreeID = Did;
    };
}
```

¹ <https://en.cppreference.com/w/c/string/byte/strcpy>

² <https://en.cppreference.com/w/c/string/byte/strncpy>